



Resumen y compresión de contexto

Cuando una conversación crece, ocurren tres cosas:

1. el prompt se hace enorme (coste/latencia),
2. aparece ruido (pierdes foco),
3. el modelo empieza a "olvidar" lo relevante porque no lo ve de forma compacta.

Una estrategia simple y muy usada en sistemas reales es:

Resumen + últimos mensajes

Guardas una "memoria comprimida" del pasado y mantienes una ventana de recencia.

Objetivos

- Entender por qué el historial largo degrada.
 - Diseñar una estrategia de compresión simple.
 - Implementar resumen periódico en Python.
 - Hacer conexión entre estado, prompts y control de formato.
-

1) Dos capas de contexto: comprimido + reciente

Modelo mental:

```
contexto_final = resumen_de_historial + últimos_N_turnos
```

Eso te permite:

- mantener lo importante del pasado sin reenviar todo
 - controlar el tamaño del input
-

2) Ejemplo: Resumir "con el modelo"

```
from google import genai
from google.genai import types

MODEL = "gemini-3-flash-preview"
client = genai.Client()

def resumir(texto: str, max_puntos: int = 5) -> str:
    prompt = f"""
    Eres un asistente que resume conversaciones.

    Tarea: resume el texto en {max_puntos} puntos clave.
    Reglas:
    - no inventes datos
    - mantén nombres, decisiones y números
    - devuelve solo la lista en texto plano

    Texto:
    {texto.strip()}
    """.strip()

    r = client.models.generate_content(
        model=MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0.2),
    )
    return (r.text or "").strip()
```

Este resumen se convierte en un objeto de estado, como cualquier otro:

```
conversation_summary = resumir(conversation_text)
```

3) Patrón de resumen periódico

Una estrategia didáctica:

- guardas todos los turnos en **history**
- cada K turnos, actualizas **summary**
- para preguntar, envías: **summary + últimos_N_turnos**

Ejemplo de resumen periódico:

```
from google import genai
from google.genai import types

MODEL = "gemini-3-flash-preview"
client = genai.Client()
```

```

history: list[types.Content] = []
summary: str = ""

WINDOW = 8
RESUMIR_CADA = 12

def ask_with_summary(user_text: str) -> str:
    global summary

    # 1) añade el turno del usuario al historial "completo"
    history.append(types.Content(role="user", parts=
[types.Part.from_text(text=user_text)]))

    # 2) decide si toca resumir (por tamaño o por número de turnos)
    if len(history) >= RESUMIR_CADA and len(history) % RESUMIR_CADA == 0:
        full_text = "\n".join(
            f"{c.role}: {c.parts[0].text}" for c in history if c.parts and
c.parts[0].text
        )
        summary = resumir(full_text, max_puntos=6)

    # 3) construye el contexto final (resumen + ventana reciente)
    recent = history[-WINDOW:]

    contents: list[types.Content] = []
    if summary:
        contents.append(
            types.Content(
                role="user",
                parts=[types.Part.from_text(text=f"Resumen de la conversación
hasta ahora:\n{summary}")],
            )
        )

    contents.extend(recent)

    # 4) llama al modelo con el contexto final
    r = client.models.generate_content(model=MODEL, contents=contents)
    answer = (r.text or "").strip()

    # 5) guarda respuesta del modelo en el historial "completo"
    history.append(types.Content(role="model", parts=
[types.Part.from_text(text=answer)]))
    return answer

```

Qué enseña este patrón:

- el historial "completo" es tu base de verdad (estado),
- el contexto enviado es una **vista** (selección + compresión).

4) Compresión como problema de diseño (qué entra en el resumen)

Un resumen útil debe capturar:

- hechos: nombres, datos, restricciones
- decisiones: "se eligió X"
- objetivos actuales: "ahora queremos Y"
- preferencias del usuario

Un resumen malo mete:

- relleno conversacional
- frases vagas ("hablamos de cosas")
- detalles que no se usan

En software real, el resumen es un "objeto" que tu app controla. Si lo necesitas estructurado, puedes pedir JSON (continuidad Unidad 02) y validarlo.

5) Errores comunes

1. Resumir demasiado pronto (pierdes detalles).
 2. Resumir demasiado tarde (ya te pasaste de tamaño y latencia).
 3. Resumen sin reglas (se vuelve genérico y poco útil).
 4. Olvidar que el resumen también consume tokens (debe ser breve).
-

Resumen

- Resumen = memoria comprimida; ventana reciente = recencia.
- `summary + últimos_N_turnos` es un patrón simple y potente.
- El contexto enviado es una selección del estado: no tienes que reenviar todo.